Onsite engineering docs

Audience: people working on Onsite. · **Updated:** 2026-10-30 · **Covers:** v0.1.2 (branch `m1`)

These docs explain how Onsite works today: what it does, why, and how the code is laid out. They describe the app **as built**. Ideas that are not built yet are marked **Planned** or **Open question**.

Two audiences, one source:

- **People:** read the PDFs in `pdf/`. They are a snapshot, rebuilt on purpose when a release or a big change makes them stale.
- **AI agents:** read these `.md` files. They are kept in step with the code, commit by commit, and may become denser over time than the PDFs.

Reading order

#	Document	Read it when you want to know…
1	ARCHITECTURE	How the pieces fit: processes, modules, data flow, events. **Start here.**
2	PRD-01 Product	What Onsite is for, who uses it, the rules it never breaks, what is shipped.
3	PRD-02 Interview loops	Rounds, panels, temperaments, feedback and the loop verdict.
4	PRD-03 Voice	Spoken interviews: what the candidate hears and says.
5	PRD-04 Company research	Dossiers and company-specific loops.
6	PRD-05 Resume check	The ATS check and the AI screen.
7	PRD-06 Coding workspace	Editor, Run/Submit, whiteboard, notes and the question bank.
8	PRD-07 Settings, privacy, updates	Bring-your-own-AI, appearance, storage, updates.
9	DD-01 AI brain	How model calls work on five providers.
10	DD-02 Interview engine	A round from "Join" to the report, in code.
11	DD-03 Voice pipeline	TTS batching, STT segments, Whisper, barge-in.
12	DD-04 Research and dossiers	The research job and the dossier format.
13	DD-05 Resume check	Text extraction, checks, scores and the ATS profiles.
14	DD-06 Question bank and runner	The CSV bank, drivers per language, Run/Submit.
15	DD-07 Storage and data	SQLite schema, settings keys, files on disk, retention.

#	Document	Read it when you want to know…
16	DD-08 Release and updates	CI, installers, the in-app updater.
17	UI design	Where UI code lives, the design language, every screen.

PRD = product requirements: the *what* and *why*. **DD** = design doc: the *how*. Every PRD links to the DDs that build it.

Conventions

- Requirement ids look like `LOOP-R3`. They are stable: refer to them in commits and issues.
- Status words: **Shipped** (in v0.1.2), **Partial** (works with known gaps), **Planned**, **Open question**.
- Code is referenced as `path:line` from the repo root, for example `src-tauri/src/lib.rs:53`. Line numbers drift; the function name next to them is the stable part.
- Diagrams are Mermaid. GitHub renders them; so does the PDF build.

Building the PDFs

```
cd docs/engineering/tools
npm install          # once: marked, mermaid, playwright-core
npm run build        # all docs → docs/engineering/pdf/*.pdf
npm run build -- UI-DESIGN.md   # just one
```

The builder uses Playwright's cached Chromium, an installed Chrome/Edge, or `CHROME=/path/to/chrome`. The Markdown files are the source of truth. A rebuild renders whatever they say now, so read the new PDFs before committing them.

Keeping these docs true

- Change the `.md` in the same commit as the code that makes it wrong. Rebuild the PDFs when it matters for people.
- `CLAUDE.md` holds the dense, verified facts (flag names, measured latencies). These docs explain them. When the two disagree, check the code, then fix whichever is wrong.
- If the graphify graph exists (`graphify-out/`), `graphify query "<question>"` is a quick way to find where something lives before you update a doc.